Build Guide

Automatic Chain-Stitch Crochet Machine (ESP32, Wi-Fi controlled)

---

1. Overview

This guide explains how to build a machine that produces a crochet chain stitch automatically and is started, stopped and monitored from a phone browser over Wi-Fi. One hook is moved up and down by a stepper-driven lead screw, a servo rotates the hook to latch the yarn, and a second servo sweeps a yarn arm around the hook.

Scope: this design makes chain stitches only. True single crochet needs the work to be fed and rotated between stitches, which is a larger mechanism. Treat this build as the foundation and get chain stitching reliable first.

Status: the code and wiring follow standard ESP32, AccelStepper and ESP32Servo practice, but they have not been run on your hardware. Expect to calibrate positions and angles in Section 7; the mechanical timing is where most of the effort goes.

2. Parts and Tools

The complete parts list with quantities, estimated prices and purchase links is in the separate Bill of Materials document (Crochet_Machine_BOM.docx / .pdf / .xlsx). Estimated total including 10% contingency: Rs 6,017.

Tools required:

- Soldering iron and solder, multimeter, wire stripper
- Screwdriver set and hex keys (M3, M4, M5)
- Access to a 3D printer, or a print service, for the brackets
- Laptop with Arduino IDE 2.x and a USB cable

3. Safety

- Disconnect power before changing any wiring. Never connect or disconnect a stepper motor while the driver is powered, as this can destroy the driver.
- Keep fingers clear of the hook and yarn arm while the machine is running. Test with the hook above the working area first.
- Set the LM2596 output to 5.0 V with a multimeter before connecting the servos.
- Insulate all exposed connections on the 12 V side.

4. Mechanical Assembly

1. Mount the NEMA 17 motor to a printed motor bracket on the base, with the shaft pointing up.
2. Join the lead screw to the motor shaft using the 5 mm to 8 mm flexible coupler. The flexible coupler absorbs small misalignment.
3. Fix the two 8 mm smooth rods parallel to the lead screw, one on each side, using printed or aluminium end mounts.
4. Build the carriage: a printed block holding the brass nut and the two LM8UU bearings, so it slides on the rods and rises or falls as the screw turns.
5. Mount servo 1 (hook servo) on the carriage and attach the crochet hook to its horn so the servo can rotate the hook to open or latch it.

6. Mount servo 2 (yarn arm servo) on the carriage beside the hook, with a short arm whose tip can sweep a half circle around the hook.
7. Install the limit switch at the top of the travel so the carriage presses it when fully raised. This is the home position.
8. Fix a small yarn guide (an eyelet or tube) above the arm so the yarn feeds smoothly with light tension.

5. Electronics and Wiring

Pin assignments used in the code:

Signal	ESP32 GPIO	Connects to
STEP	26	A4988 STEP
DIR	27	A4988 DIR
ENABLE	25	A4988 EN (LOW = driver on)
Hook servo signal	18	MG90S #1 signal wire
Yarn arm servo signal	19	MG90S #2 signal wire
Limit switch	33	Switch COM to GPIO 33, switch NO to GND (internal pull-up used)

Power connections:

- 12 V adapter positive to the A4988 VMOT and to the LM2596 IN+; adapter negative to the A4988 GND and LM2596 IN-.
- Place a 470 uF capacitor across A4988 VMOT and GND, close to the driver, with correct polarity.
- LM2596 OUT+ (5.0 V) to both servo power wires; OUT- to servo grounds.
- Join all grounds together: 12 V supply, 5 V rail and ESP32 GND. Without a common ground the signals will not work.
- A4988 VDD to ESP32 3V3 and logic GND to ESP32 GND. Tie A4988 RESET and SLEEP together. Leave MS1 to MS3 unconnected for full-step mode.
- Connect the stepper coil pairs to the A4988 outputs 1A/1B and 2A/2B. Use the multimeter in continuity mode to identify the two coil pairs.
- Power the ESP32 from USB while developing. Do not power servos from the ESP32 5 V pin.

Set the A4988 current limit: with the motor connected, turn the potentiometer slowly while checking the motor runs smoothly without overheating. A starting point is a reference voltage of about 0.8 V for a 1.5 A motor (the exact formula depends on your module; check its datasheet).

6. Software Setup

1. Install Arduino IDE 2.x from arduino.cc.
2. Open File > Preferences and add the ESP32 board package URL from Espressif, then install "esp32" in Boards Manager.
3. Select Tools > Board > ESP32 Arduino > ESP32 Dev Module.
4. In Library Manager install "AccelStepper" (Mike McCauley) and "ESP32Servo" (Kevin Harrington).
5. Open the provided file crochet_machine.ino (or paste the code from Section 10).
6. Edit WIFI_SSID and WIFI_PASS to your 2.4 GHz network. The ESP32 does not support 5 GHz Wi-Fi.

7. Connect the ESP32 by USB, select the correct port, and click Upload. If upload stalls, hold the BOOT button while it says "Connecting".
8. Open Serial Monitor at 115200 baud. Note the printed IP address.

7. Calibration and First Run

Test each part separately before running a full stitch cycle.

1. Homing: on power-up the carriage should move toward the limit switch and stop. If it moves away from the switch, swap the sign of setSpeed(-400) to +400 or reverse one motor coil pair. If homing fails, the Serial Monitor prints "Homing failed".
2. Hook servo: confirm that HOOK_OPEN (90) and HOOK_LATCH (0) open and close the hook properly. Adjust the angles in the constants at the top of the sketch.
3. Yarn arm: confirm that the sweep from ARM_PARK to ARM_WRAP_END passes fully around the hook without hitting it. Adjust the two constants.
4. Depth: set Z_DOWN (steps from home to the loop). In full-step mode a 200-step motor with an 8 mm lead screw moves 8 mm per 200 steps, so 600 steps is about 24 mm. Increase or decrease until the hook reaches the loop without hitting the base.
5. Single stitch: open the IP address in a phone browser on the same Wi-Fi, enter 1 and press Start. Watch one full cycle slowly and adjust.
6. Run 20 stitches. Tune the servo delays (8 ms per step in the arm sweep, 250 ms latch, 150 ms release) until the chain forms consistently.

8. Web Controls

- Start: runs the entered number of stitches.
- Stop: halts after the current stitch completes.
- Home: re-runs the homing routine. Use this if the carriage was moved by hand.
- The status line shows stitches done, the target, and whether the machine is homed.
- JSON status is also available at /status for use in other apps.

9. Troubleshooting

Problem	Likely cause and fix
Motor vibrates but does not turn	Coil wires mismatched. Identify coil pairs with a multimeter. Also raise the A4988 current limit slightly.
Motor skips steps or stalls	Reduce setMaxSpeed or setAcceleration. Check the lead screw for binding and the coupler for slip.
ESP32 resets when servos move	Servos are drawing from the wrong rail or the 5 V rail sags. Power them from the LM2596 and add a 470 uF capacitor on the servo rail.
Servos jitter	Missing common ground, or a noisy supply. Join all grounds and add the capacitor.
Homing never completes	Limit switch wired to NO and GND? Check with the multimeter. The pin reads HIGH until pressed.
Cannot open the web page	Phone and ESP32 must be on the same 2.4 GHz network. Confirm the IP in Serial Monitor.
Yarn does not catch on the hook	Adjust yarn arm sweep and the HOOK_LATCH angle. Use smooth cotton yarn and keep tension light.
Stitches are uneven	Reduce speed, add delays, and check the carriage slides freely on the rods without wobble.

10. Source Code

The same code is supplied as crochet_machine.ino. Note: the Stop button takes effect after the stitch in progress completes.

```
// Automatic chain-stitch crochet machine - ESP32
// Libraries: AccelStepper, ESP32Servo (install via Library Manager)
// Board: ESP32 Dev Module

#include <WiFi.h>
#include <WebServer.h>
#include <AccelStepper.h>
#include <ESP32Servo.h>

const char* WIFI_SSID = "YOUR_WIFI";
const char* WIFI_PASS = "YOUR_PASSWORD";

#define STEP_PIN  26
#define DIR_PIN   27
#define EN_PIN    25
#define HOOK_PIN  18
#define ARM_PIN   19
#define LIMIT_PIN 33

// ---- Calibration values (tune on your machine) ----
const long Z_DOWN = 600;             // steps from home to loop depth (full-step mode)
const int  HOOK_OPEN = 90;           // servo angle, hook open
const int  HOOK_LATCH = 0;           // servo angle, hook latched on yarn
const int  ARM_PARK = 0;             // yarn arm parked
const int  ARM_WRAP_END = 180;       // yarn arm end of sweep
const unsigned long HOME_TIMEOUT_MS = 20000;

AccelStepper z(AccelStepper::DRIVER, STEP_PIN, DIR_PIN);
Servo hook, arm;
WebServer server(80);

int target = 0, done = 0;
bool running = false;
bool homed = false;

bool homeZ() {
  digitalWrite(EN_PIN, LOW);
  z.setSpeed(-400);
  unsigned long t0 = millis();
  while (digitalRead(LIMIT_PIN) == HIGH) {
    z.runSpeed();
    if (millis() - t0 > HOME_TIMEOUT_MS) {
      digitalWrite(EN_PIN, HIGH);
      return false;                  // switch never triggered
    }
  }
  z.setCurrentPosition(0);
  return true;
}

void moveZ(long pos) {
  z.moveTo(pos);
  int n = 0;
  while (z.distanceToGo() != 0) {
    z.run();
    if (++n % 200 == 0) server.handleClient();   // keep web server responsive
```

```
  }
}

void stitch() {
  hook.write(HOOK_OPEN);
  arm.write(ARM_PARK);
  moveZ(Z_DOWN);                                  // hook into loop
  for (int a = ARM_PARK; a <= ARM_WRAP_END; a += 5) {
    arm.write(a);                                 // wrap yarn
    delay(8);
  }
  hook.write(HOOK_LATCH);                         // latch yarn
  delay(250);
  moveZ(0);                                       // pull through loop
  hook.write(HOOK_OPEN);                          // release
  delay(150);
  arm.write(ARM_PARK);
}

void handleStart() {
  if (!homed) { server.send(409, "text/plain", "Not homed. Check limit switch, then open /home");
return; }
  target = server.hasArg("n") ? server.arg("n").toInt() : 10;
  done = 0;
  digitalWrite(EN_PIN, LOW);
  running = true;
  server.send(200, "text/plain", "Started " + String(target) + " stitches");
}

void handleStop() {
  running = false;
  server.send(200, "text/plain", "Stopped");
}

void handleHome() {
  running = false;
  homed = homeZ();
  server.send(200, "text/plain", homed ? "Homed" : "Homing failed");
}

void handleStatus() {
  server.send(200, "application/json",
    String("{\"running\":") + (running ? "true" : "false") +
    ",\"homed\":" + (homed ? "true" : "false") +
    ",\"done\":" + String(done) +
    ",\"target\":" + String(target) + "}");
}

void handleRoot() {
  server.send(200, "text/html",
    "<meta name=viewport content='width=device-width,initial-scale=1'>"
    "<h2>Crochet Machine</h2>"
    "<input id=n type=number value=20> "
    "<button onclick=\"fetch('/start?n='+n.value)\">Start</button> "
    "<button onclick=\"fetch('/stop')\">Stop</button> "
    "<button onclick=\"fetch('/home')\">Home</button>"
    "<p id=s></p><script>setInterval(async()=>{let r=await (await fetch('/status')).json();"
    "s.innerText=r.done+' / '+r.target+(r.running?' (running)':'')+(r.homed?'':' - not
homed');},1000)</script>");
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(EN_PIN, OUTPUT);
  digitalWrite(EN_PIN, HIGH);                    // driver disabled until needed
  pinMode(LIMIT_PIN, INPUT_PULLUP);

  z.setMaxSpeed(1200);
  z.setAcceleration(800);

  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  hook.setPeriodHertz(50);
  arm.setPeriodHertz(50);
  hook.attach(HOOK_PIN, 500, 2400);
  arm.attach(ARM_PIN, 500, 2400);
  hook.write(HOOK_OPEN);
  arm.write(ARM_PARK);

  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) { delay(300); Serial.print("."); }
  Serial.println();
  Serial.print("Open http://");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/start", handleStart);
  server.on("/stop", handleStop);
  server.on("/home", handleHome);
  server.on("/status", handleStatus);
  server.begin();

  homed = homeZ();
  Serial.println(homed ? "Homed" : "Homing failed - check limit switch");
}

void loop() {
  server.handleClient();
  if (running && done < target) {
    stitch();
    done++;
  } else if (running) {
    running = false;
    digitalWrite(EN_PIN, HIGH);
  }
}
```

11. Next Steps

- Add a rotating or advancing work holder so the machine can build rows of single crochet.
- Add a yarn tension sensor or a break detector to stop automatically when yarn runs out.
- Switch to a TMC2209 driver and 1/16 microstepping for quieter, smoother motion (remember to multiply Z_DOWN and speeds accordingly).
- Add OTA firmware updates so the code can be changed over Wi-Fi.